

Gila (Geetha) Nehemia

gnehemia@gmail.com • linkedin.com/in/gnehemia

Portfolio case study

Enterprise release notes automation pipeline

A full project lifecycle, from pain point discovery to a working AI-powered content operations system, for complex, multi-phase software and hardware releases

Pre-production

pilot complete

A documentation pipeline that previously required significant manual effort across every release cycle now runs automatically, reducing manual production time by over 70%. The structured pilot is complete, with formal metrics collected, and the pipeline has moved into pre-production.

HOW IT STARTED

This wasn't a top-down AI initiative. It started with one person solving her own problem.

"I wasn't trying to build an AI pilot. I was trying to solve my own problem. The work opened conversations with senior leadership, generated demo requests across teams, and connected me to the organization's broader AI initiatives, all because the solution spoke for itself."

The pilot created organic momentum that no formal program produced. It was built from real frustration, real expertise, and a genuine need to do better work.

Senior leadership conversations

Work surfaced to manager's manager organically through results

Cross-team AI pilot connection

Linked to broader organizational AI initiatives led by another manager

Demo requests

Colleagues and stakeholders asked to see the work firsthand

ORIGIN: THE FORMAL FOUNDATION

A year before the first line of automation was built, a colleague and I identified that the release notes process was fundamentally broken. We didn't jump to a solution. We did it properly.

Step 1: Discovery

Pain point identification

Collaboratively identified and documented the specific friction points: manual effort, inconsistency, bottlenecks, and quality risk across multiple systems and product lines.

Step 2: Documentation

Pain points formally documented

Pain points were written up and shared, creating a documented case for change and an agreed baseline to measure improvement against.

Step 3: Planning

Project artifacts produced

Before any automation was built, a full set of governance documents were created: design documentation, phased pilot plan, and testing plan.

Step 4: Build

Pipeline designed and built

With the foundation in place, the six-skill modular pipeline was manually architected and built, grounded in the documented pain points and governed by the project artifacts.

THE PROBLEM

Enterprise release notes for large-scale software products are manually intensive, inconsistent across product lines, and difficult to scale across service pack releases and hardware integrations. Each release cycle required significant human effort across multiple enterprise systems with no automated handoffs between them.

THE INSIGHT

My own release note workflow, built over years of enterprise documentation experience, could be deconstructed into discrete, testable steps and encoded into an AI pipeline. Rather than starting from scratch, I digitized my own expertise into a replicable, automated system that applies institutional knowledge as a guardrail at every step.

ARCHITECTURE

The pipeline is built as 6 modular, independently testable skills, each consuming the output of the one before it. A shared content generation engine is called by multiple skills, applying institutional style guide and terminology guardrails consistently. Example XML structures are embedded directly within each skill, enabling the system to generate correctly structured output files from scratch.

Designing each skill as a small, discrete unit means errors can be isolated at the module level without disrupting the rest of the pipeline. The same principle used in software engineering. Build once, reuse everywhere.

ENGINEERING PROGRESSION

The pipeline was built following a deliberate engineering arc, the same progression used by professional AI engineering teams:

→ **Deterministic model**

Built first. Rule-based, predictable, controlled outputs. Every input produces the same output. The foundation was made reliable before any intelligence was layered on top.

→ **JSON logging and observability**

The pipeline is fully instrumented. Every action is logged, traceable, and auditable, enabling debugging, improvement, and future metrics collection.

→ **Python processing**

Connective tissue that moves structured data intelligently between enterprise systems at each stage of the pipeline.

→ **Guardrails**

AI behavior is constrained by institutional knowledge: style guide, terminology file, and embedded XML templates, keeping quality consistent and outputs compliant.

→ **Agentic reasoning**

The system now reasons about context across interactions. During testing, the model identified a version mismatch unprompted and asked whether to extend logic to the next release, anticipating the next step without being instructed to.

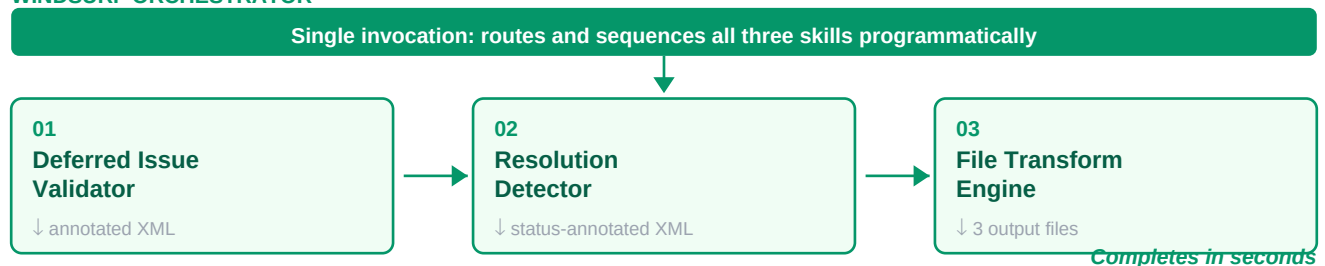
→ **Programmatic CCMS integration**

In progress. Direct API access to the enterprise content management system will eliminate the final manual handoff, completing a fully continuous pipeline from source data to published documentation.

ORCHESTRATOR LAYER

Three of the pipeline's core data-processing skills are linked and sequenced by a Devin-native orchestrator, a coordination layer that sits above the modular skills and manages execution, state handoff, and routing programmatically. Rather than triggering each skill manually, the orchestrator fires all three in a single automated run, passing structured output from each skill directly to the next.

WINDSURF ORCHESTRATOR



How Devin enables this

Devin's MCP server integration gives the orchestrator direct, programmatic access to the engineering issue tracking system and the file system. Each skill runs as an instrumented unit and the orchestrator reads its JSON-logged output to determine routing for the next skill, with no manual handoff and no context switch between tools.

What the orchestrator coordinates

■ **Skill 01. Deferred issue validator:** queries the issue tracker, cross-references the master XML, flags deferred issues, and generates first-draft content

■ **Skill 02. Resolution detector:** scans for resolved issues and annotates the XML with fixed-status comments

■ **Skill 03. File transformation engine:** rebuilds known issues and generates the resolved issues and new features XML files

Three enterprise-system queries, two XML transformations, and three structured output files, triggered from a single Devin orchestrator invocation and completing in seconds.

THE SIX SKILLS

01 Deferred issue validator

Queries the engineering issue tracking system using standard engineering queries to identify deferred issues. Cross-references the master XML file, flags new deferred issues, and generates compliant first-draft documentation using the shared content generation engine.

■ uses content generation engine ↓ annotated XML ↓ first-draft content

02 Resolution detector

Scans the engineering issue tracking system and enterprise knowledge base for resolved issues. Annotates the XML file with fixed-status comments, transforming it into a live document that reflects current release state at all times.

↓ status-annotated XML

03 File transformation engine

Rebuilds the known issues file so only current deferred issues remain. Simultaneously generates two new XML files from scratch using embedded template structures: one for resolved issues, one for new features processing.

↓ rebuilt known issues XML ↓ resolved issues XML ↓ new features XML

04 **New features extractor**

Simultaneously queries the engineering issue tracking system and design platform to pull feature data, bridging engineering tickets and design artifacts automatically. Passes structured data to the content generation engine for compliant first-draft content.

■ uses content generation engine ↓ completed new features XML ↓ first-draft content

05 **Content generation engine**

Shared service used across the pipeline. Takes structured data and generates compliant first-draft content using the institutional style guide and terminology file as guardrails, producing on-brand, accurate documentation before a human ever touches it.

■ shared service · called by skills 01 and 04

06 **SME notification engine**

Automatically posts a comment in the engineering issue tracking system to the responsible subject matter expert when a new deferred issue is documented, closing the loop between automation and human review without manual tracking.

↓ automated SME notification

WHAT THE PIPELINE PRODUCES

Five outputs from one pipeline run, all compliant, correctly structured, and routed for review.

Output 01	Output 02	Output 03	Output 04	Output 05
Rebuilt known issues XML	Resolved issues XML	New features XML	First-draft content for all sections	Automated SME review notification

ACTIVE ADVOCACY

The pipeline is a proof of concept for something bigger. We are actively advocating for a formal, organization-wide process with engineering to make this the standard, not a workaround.

The case we are making to engineering

A formally integrated automation workflow between engineering and documentation would deliver measurable business outcomes beyond efficiency, connecting documentation quality directly to customer experience and support cost reduction.

→ Automate the full documentation workflow so release notes are generated, validated, and routed without manual intervention at any stage.

→ Deliver consistently high-quality documentation that reduces customer confusion and reduces inbound service calls as a direct result.

→ Efficiently document bugs and fixes in real time, so customers always have accurate, current information about known issues and resolutions.

Documentation quality

Consistent, accurate, on-brand across every release

Service call reduction

Fewer tickets from customers who can't find answers

Engineering alignment

Bugs and fixes documented in real time, not retrospectively

Scalability

Pipeline extends to any product line without added headcount

IN-MARKET (SERVICE PACK) EXTENSION

Pre-production - built, no longer a pilot

Built collaboratively with a colleague, adapting the full release notes pipeline to automate release notes for in-market (service pack) releases: incremental updates issued after a product's initial launch. This extension has progressed beyond pilot and is now in pre-production.

Service pack cycles introduce a different shape of problem than a full release: smaller, more frequent updates that still require the same rigor around deferred issues, resolved issues, and new features tracking. Rather than building a parallel system, the in-market pipeline reuses the same modular skills and content generation engine, adapted to handle incremental change documentation at service pack cadence.

Several adjustments were required to bring the pipeline into conformance with service pack release guidelines. Working a few hours a day, we completed those adjustments within a 2-3 day period. The result: manual effort on service pack release notes is now reduced by over 70%.

TOOLS AND INTEGRATION

Python

JSON processing

XML manipulation

Template-driven generation

Devin

MCP server integration

Orchestrator layer

Engineering issue tracking

Enterprise knowledge base

Design platform integration

Enterprise CCMS

Agentic workflow design

Modular pipeline architecture